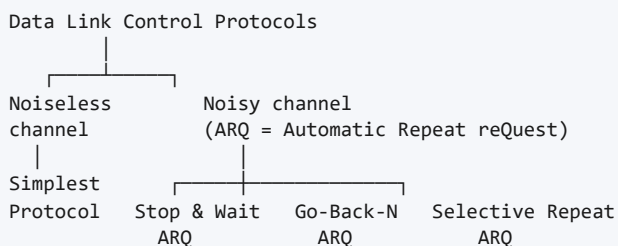


Data Link Layer: Flow Control, ARQ & Switching

PART 1 — Flow Control & ARQ Protocols

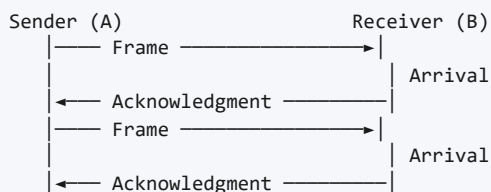
1. Classification of Data Link Control Protocols



- **Noiseless channel** — assumes no transmission errors; the only concern is not overwhelming the receiver.
- **Noisy channel (ARQ)** — assumes frames can be lost or corrupted; the sender must detect this and **automatically request retransmission**. When a receiver detects an error, it has three options: **drop the corrupted data, try to correct it** (see the Error Detection & Correction topic — Hamming code), or **request the sender resend it** — ARQ protocols are all built around this third option.

2. Stop-and-Wait (the simplest protocol, noiseless assumption)

For every single frame sent, the sender **waits** for an acknowledgment before sending the next one.

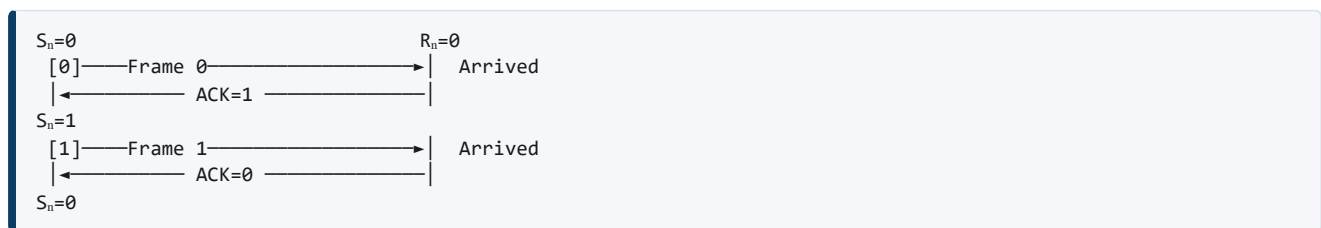


- Trivially simple, and correct by construction — the receiver is never sent more than it can handle, since nothing new arrives until the current frame is acknowledged.
- Extremely poor link utilization — the sender sits idle for a full round-trip time after every single frame, even on a fast, low-latency link.

3. Stop-and-Wait ARQ (adds error handling)

Adds a **timer** and **sequence numbers** to Stop-and-Wait, to survive a noisy channel where frames or acknowledgments can be lost or corrupted.

- A timer starts the moment a frame is sent. If the **ACK doesn't arrive before the timer expires**, the sender assumes the frame (or its ACK) was lost, and **resends the same frame**.
- **Sequence numbers**: with only one outstanding frame at a time, a **single bit (0/1)** is sufficient. S_n (the sender's next sequence number to use) increments only when an ACK is received; R_n (the receiver's next expected sequence number) increments once the correct data arrives.
- **The ACK number is always the sequence number the receiver expects next** — not the number of the frame just received. This is what lets a lost/duplicate frame be detected: if a duplicate frame 0 arrives again after already being acknowledged, the receiver recognizes it (its R_n has already moved past it) and simply re-sends the same ACK without accepting the data twice.



Why does loss recovery work with just 1 bit of sequence number? Because Stop-and-Wait only ever has **one** frame in flight at a time — there's never ambiguity about "which" frame a timeout or ACK refers to, so 0/1 alternation is always sufficient to distinguish "this is the next new frame" from "this is a duplicate of what I already have."

- **✗** Still suffers the same fundamental utilization problem as plain Stop-and-Wait — one frame in flight at a time, regardless of the link's actual capacity.

4. The Sliding Window — fixing utilization

Window = the number of frames a sender is permitted to transmit **before needing to wait** for any acknowledgment. The window "slides" forward across the sequence-number space as ACKs arrive, hence the name.

This is the mechanism both Go-Back-N and Selective Repeat build on — instead of stopping after every single frame, the sender can have **multiple frames outstanding (unacknowledged) simultaneously**, dramatically improving link utilization on high-latency or high-bandwidth links.

5. Go-Back-N ARQ

- **Window size** = $2^m - 1$ (fixed), where m is the number of bits used for sequence numbers — this specific upper bound (not 2^m) exists to avoid ambiguity between a new window and an old one when sequence numbers wrap around.
- The sender can transmit up to the entire window's worth of frames without waiting, but the **receiver only ever accepts frames in strict order** — it has **no buffer** for out-of-order frames.



Why does one lost frame force retransmitting everything after it? Since the receiver has no buffer for out-of-order arrivals, it must **discard every subsequent frame** once one is lost — even though frames 3 and 4 physically arrived intact — because it can't safely deliver them to the upper layer out of sequence. When the sender's timer for frame 2 expires, it must therefore resend frame 2 **and everything sent after it** (3, 4, ...), since the receiver discarded all of them.

- **ACKs are cumulative** — an ACK for frame n means "I have correctly received everything up through frame $n-1$ and am now expecting frame n ." This lets a single ACK implicitly acknowledge a whole run of frames at once.
- Much better utilization than Stop-and-Wait — multiple frames in flight.
- Wasteful on lossy links — a single lost frame forces retransmission of every frame sent after it, even ones that arrived successfully.

6. Selective Repeat ARQ

Fixes Go-Back-N's main waste: the receiver **does** buffer out-of-order frames, and the sender retransmits **only the specific frame(s) that were actually lost/corrupted** — not everything after them.

- The receiver sends a **NAK** (negative acknowledgment) for a specifically missing/corrupted frame; the sender resends **just that frame**.
- **Window size at both sender and receiver** must be equal, and specifically bounded to $2^{(m-1)}$ (half of Go-Back-N's) — a smaller bound than Go-Back-N, needed specifically to prevent the receiver from confusing a retransmitted old frame with a new one when sequence numbers wrap around, now that out-of-order frames are being buffered.
- Much more bandwidth-efficient than Go-Back-N on lossy links — no wasted retransmission of already-successfully-received frames.

- ✗ More complex to implement — requires buffering at the receiver and more careful bookkeeping of which specific frames are missing.



7. Piggybacking

Since data flow between two communicating parties is usually **bidirectional**, a frame carrying data from A to B can also carry **control information (an ACK) about frames received from B** in the same frame — instead of sending a separate, dedicated ACK frame.

- ✓ Saves bandwidth and reduces the number of frames sent overall (one frame does double duty).
- Works naturally with S_n/R_n -style sequence-number schemes already discussed, since the ACK field is just another field in an otherwise normal data frame.

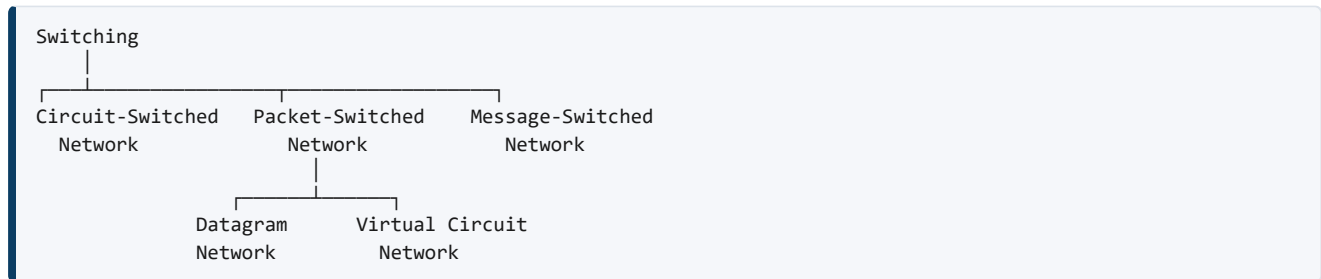
Summary Comparison — ARQ Protocols

Protocol	Frames in flight	Receiver buffers out-of-order?	On loss, retransmits	Window size
Stop-and-Wait ARQ	1	N/A	Just that 1 frame	1
Go-Back-N ARQ	Up to $2^m - 1$	No	The lost frame and everything after it	$2^m - 1$
Selective Repeat ARQ	Up to $2^{(m-1)}$	Yes	Only the lost/corrupted frame	$2^{(m-1)}$

PART 2 — Switching

8. Why Switching Exists

A network connecting many devices can't feasibly give every pair of devices a dedicated physical link (recall the mesh topology cabling explosion from the Topologies topic). **Switching** is how a network of shared links and intermediate nodes moves data from a specific source to a specific destination.



9. Circuit-Switched Network

A **dedicated physical/logical path** is established between sender and receiver **before** any data transmission, and that path remains reserved exclusively for that session until it's explicitly torn down.

Three phases: connection establishment → data transfer (over the now-fixed path) → connection termination.

- **✓ Predictable delay and quality** — since bandwidth is fixed and dedicated, and the path doesn't change mid-session, performance is highly consistent (crucial for real-time voice/video).
- **✗ Inefficient resource use** — the reserved bandwidth is held for the entire session **even during idle periods** (e.g., silence during a phone call), and the setup/teardown itself introduces delay.
- **Classic example:** the traditional telephone network (PSTN).

10. Packet-Switched Network

Data is broken into **packets**, each carrying its own header (source/destination addressing), and routed **independently** through the network — the modern Internet's approach.

- **Store-and-forward:** each intermediate node (router/switch) receives an entire packet, briefly buffers it, and then forwards it onward — rather than the transmission happening as a continuous, uninterrupted signal.
- **Shared resources:** many different transmissions can share the same links/bandwidth, since packets from different sources are only using the link exactly when they're actually being transmitted.

-  Much more efficient use of bandwidth for bursty, sporadic traffic (most real traffic — web browsing, messaging) — no reserved-but-idle capacity.
-  **Variable delay** — since packets are routed independently and links are shared/contended, transmission delay for each packet can vary (this is what causes jitter in real-time applications over packet-switched networks).

Two sub-types:

10.1 Datagram Network

Each packet is routed **independently**, based purely on its destination address and the current state of the network at that moment — different packets from the same transmission may take **different paths**, and may even arrive **out of order**.

- This is exactly how IP itself works — connectionless, best-effort, packet-by-packet routing decisions (developed further in the Routing Protocols topic).

10.2 Virtual Circuit Network

Connection-oriented, but at the *logical* level rather than a physically dedicated circuit: a **virtual circuit** is set up before data transfer, identified by a **Virtual Circuit Identifier (VCI)**, and all packets belonging to that session follow the **same predetermined route** and are delivered **in order**.

- **Fixed routing** (decided once, at setup) + **low per-packet overhead** (routers just look up the VCI rather than making a fresh routing decision for every packet) + **guaranteed in-order delivery**.
- **Used in:** ATM (Asynchronous Transfer Mode, using VPI/VCI) and Frame Relay (using DLCI).
- Sits conceptually **between** pure circuit switching (fully dedicated, physical) and pure datagram packet switching (fully independent, per-packet) — it borrows connection-orientation and fixed-path guarantees from circuit switching, while still statistically sharing the underlying physical links like packet switching does.

11. Message Switching (largely historical)

An **entire message** (not broken into packets) is sent as one unit, store-and-forwarded hop by hop from source to destination. Predates both circuit and packet switching; used in old telex/teleprinter networks. Almost entirely superseded today by packet switching, which offers far better resource sharing and lower per-hop latency (no need to buffer an entire, potentially huge message at each intermediate hop before forwarding).

Comparison: Circuit vs Datagram vs Virtual Circuit

Aspect	Circuit-Switched	Datagram (Packet)	Virtual Circuit (Packet)
Connection setup	Required (dedicated path)	Not required	Required (logical path)
Path per packet	Same (fixed)	Can differ per packet	Same (fixed, logical)
Delivery order	N/A (continuous stream)	Not guaranteed	Guaranteed
Resource sharing	None (dedicated)	Full sharing	Shared, but path is reserved logically
Per-packet routing decision	N/A	Every packet, independently	Only once, at setup
Delay predictability	High	Low (variable)	Moderate-high
Example	PSTN (telephone)	IP / the Internet	ATM, Frame Relay

Interview soundbite: "Circuit switching reserves an entire physical path up front — predictable but wasteful if idle. Datagram packet switching makes a fresh routing decision for every single packet — efficient and resilient, but delivery order and delay aren't guaranteed. Virtual circuit switching is the middle ground: it sets up a fixed logical path once (like circuit switching's predictability) but still statistically shares physical links across many virtual circuits (like packet switching's efficiency)."

Interview Questions With Answers

Q1. Why is Stop-and-Wait ARQ inefficient on high-bandwidth or high-latency links, even though it correctly handles errors?

Answer: Stop-and-Wait only ever allows one frame to be outstanding at a time — the sender must wait for that frame's acknowledgment before sending the next. On a link with high bandwidth or high round-trip latency, the sender spends most of its time idle waiting for an ACK rather than actually transmitting, so the link's true capacity goes largely unused regardless of how fast it could otherwise carry data.

Q2. Why is a single bit sufficient for sequence numbers in Stop-and-Wait ARQ, but not in Go-Back-N or Selective Repeat?

Answer: Stop-and-Wait only ever has one unacknowledged frame in flight, so there are only ever two possibilities to distinguish at any time — "this is the next new frame" or "this is a duplicate of the one I already have" — which a single alternating bit (0/1) fully captures. Go-Back-N and Selective

Repeat allow multiple frames in flight simultaneously (a sliding window), so the sequence number must be able to distinguish among all the frames currently in the window (and avoid ambiguity with wrapped-around old sequence numbers), requiring multiple bits.

Q3. In Go-Back-N, why must the receiver discard frames 3 and 4 if frame 2 was lost, even though 3 and 4 arrived intact?

Answer: Go-Back-N's receiver has no buffer for out-of-order frames — it can only accept frames strictly in sequence. Since frame 2 never arrived, the receiver is still expecting frame 2; when frames 3 and 4 arrive ahead of it, delivering them to the upper layer out of order isn't an option (the receiver simply isn't built to hold onto them), so it discards them and waits for the sender to eventually retransmit starting from frame 2.

Q4. What specific problem does Selective Repeat solve that Go-Back-N doesn't, and what's the cost of that fix?

Answer: Selective Repeat solves Go-Back-N's wasteful retransmission of already-successfully-received frames — by buffering out-of-order frames at the receiver and having the sender resend only the specific frame(s) that were actually lost or corrupted (via NAKs), rather than everything sent after the lost frame. The cost is implementation complexity: the receiver needs buffer space to hold out-of-order frames and logic to track exactly which frames are missing, and the window size must be more tightly bounded (2^{m-1}) instead of Go-Back-N's ($2^m - 1$) to avoid the receiver confusing a retransmitted old frame with a new one after sequence numbers wrap around.

Q5. What is piggybacking, and why does it make sense specifically in a bidirectional communication scenario?

Answer: Piggybacking attaches acknowledgment information for frames received in one direction onto a data frame being sent in the *other* direction, instead of sending a dedicated, data-free ACK frame. It makes sense in bidirectional communication specifically because both parties are already sending data frames to each other anyway — riding the ACK along with outgoing data avoids the overhead of a whole separate frame (with its own header/framing overhead) whose only purpose would otherwise be carrying a small ACK field.

Q6. Why does virtual circuit switching require a setup phase, but datagram switching doesn't?

Answer: Virtual circuit switching commits to a single, fixed logical path for the entire session and identifies it with a Virtual Circuit Identifier that every router along that path needs to recognize — this mapping has to be established and configured at every intermediate node before any data can flow, which is exactly the connection-setup phase. Datagram switching makes an independent routing decision for every single packet based purely on its destination address, with no notion of a pre-agreed path or per-session state at intermediate routers, so there's nothing that needs to be set up in advance.

Q7. Why can circuit-switched networks offer more predictable delay than packet-switched (datagram) networks?

Answer: In circuit switching, the entire path's bandwidth is reserved exclusively for that session for its full duration, so once the connection is established, data flows continuously without contending with other traffic for the same links — delay is essentially fixed by the physical path. In datagram packet switching, packets share links with other traffic and are routed independently, so each packet's actual delay depends on current congestion and queuing at each hop along whatever path it happens to take — this variability (and the fact that different packets from the same flow might even take different paths) is exactly what makes delay unpredictable/variable (jitter) in packet-switched networks.

Q8. Scenario: You're designing a network for real-time video conferencing where dropped or wildly out-of-order packets are more damaging to user experience than moderately higher setup latency. Would you lean toward datagram or virtual-circuit-style packet switching, and why?

Answer: Virtual-circuit-style switching — its guaranteed in-order delivery and fixed, predetermined path per session directly address the stated priority (avoiding out-of-order delivery matters more than minimizing setup time). A pure datagram approach could route packets from the same video stream over different paths with different delays, causing packets to arrive out of order and requiring reordering buffers at the receiver (adding latency and complexity) — exactly the failure mode the scenario says is most damaging. The one-time setup cost of establishing a virtual circuit is a reasonable trade-off given that priority, especially since a video conferencing session is typically long-lived relative to that setup delay.